Entregable 5
Cloud Computing (AWS)

HERRERO CHRISTIAN

En esta entrega continúo con la arquitectura de los trabajos anteriores (Módulo 3 y Entregable 4): una plataforma web para estudiantes de un colegio secundario técnico. Los estudiantes entran por un Application Load Balancer, que reparte el tráfico entre dos servidores EC2 (t3.micro) en dos zonas de disponibilidad con Auto Scaling, y cuando hay un nuevo registro se dispara un evento por SNS que activa una Lambda con el email de bienvenida. Los archivos pesados (entregas, estáticos y logs) ya están resueltos en S3. Lo que faltaba era dónde guardar los datos de la aplicación (estudiantes, cursos, entregas y notas), y ahí aparece la comparación entre Amazon RDS y Amazon DynamoDB.

1- Requisitos de datos de la aplicación:

Primero analicé qué datos necesita guardar la plataforma y qué características tiene cada uno.

Dato	que guarda	estructura y relaciones	volumen / escalabilidad	consistencia
Perfiles de estudiantes	Nombre, email, curso, año	Fija (siempre los mismos campos). Cada estudiante pertenece a un curso y tiene muchas entregas	Bajo y predecible (cientos de estudiantes)	Importante: un email no puede repetirse, ni un dni, también salvando que no sean valores nulos o incorrectos.
Docentes,	Nombre del	Fija. Un	Muy bajo,	Importante,

cursos y materias	curso, materia, docente a cargo	curso tiene varias materias y cada materia tiene un docente	cambia una vez al año/ cada ciclo lectivo. (contando las mesas de verano)	manteniendo bien la gestion de relaciones 1:1, 1:N y N:M
Entregas (datos de cada entrega)	Estudiante, materia, fecha, estado	Fija. Une estudiante + materia + archivo	Moderado, con picos cerca de fechas de entrega	Importante
Notas y devolucion es del docente	Nota, comentario, fecha	Fija. Cada nota pertenece a una entrega	Moderado	con validaciones de notas iguales o mayores a 0, hasta 10, la nota no puede quedar guardada a medias
Archivos (PDF, .zip, videos, imágenes)	el propio archivo de cada tipo	No aplica, van directo a S3	Pesado (hasta varios GB) mayormente en carga de vídeos 4K.	no aplica

Conclusión del análisis:

- Mis datos tienen una estructura fija que no cambia seguido y están muy relacionados entre sí (estudiante, curso, materia, entrega, nota).
- Para mostrar una pantalla como "las entregas de 4.º año con el nombre del estudiante y su nota" necesito unir varias tablas (JOINs).
- El volumen es moderado y predecible (un colegio, no millones de operaciones por segundo).
- Los archivos pesados no van a la base de datos: van a S3 y en la base guardo

solamente la clave (el "nombre") del objeto.

2- Tabla comparativa: RDS vs. DynamoDB:

Para RDS tomo el motor MySQL, y para DynamoDB su modo habitual de uso.

Aspecto	Amazon rds	Amazon DynamoDB	Análisis de mi caso
Tipo de base	Relacional (tablas con filas y columnas)	No relacional (clave-valor / documentos)	RDS: mis datos son tablas relacionadas
Estructura de datos	Esquema fijo definido de antemano; cada tabla tiene sus columnas	Esquema flexible: cada elemento puede tener atributos distintos	RDS: mi estructura es fija y no necesito flexibilidad
Relaciones y JOINs	Soporta JOINs, claves primarias y claves foráneas	No soporta JOINs; hay que duplicar datos o hacer varias consultas desde la aplicación	RDS: necesito unir estudiantes, cursos, entregas y notas
Consultas	SQL: se puede consultar por cualquier columna	Se consulta por la clave definida; otras consultas son limitadas	RDS: los docentes buscan por curso, materia, fecha, estudiante, etc
Escalabilidad	Se escala cambiando el tamaño de la instancia (vertical) y con réplicas de lectura; tiene un límite	Escala de forma automática y casi ilimitada (serverless)	mi tráfico es chico, no necesito escala masiva
Rendimiento	Muy bueno con índices; depende del tamaño de instancia elegido	Milisegundos constantes aun con millones de operaciones por segundo	con mi volumen RDS rinde de perfectamente

Consistencia	Consistencia fuerte con transacciones (ACID): o se guarda todo o no se guarda nada	Por defecto eventual (puede tardar en reflejarse); se puede pedir lectura fuerte	RDS: las notas necesitan ser siempre exactas
Costo	Se paga la instancia por hora y el almacenamiento, aunque haya poco uso (una db.t3.micro es la opción más económica)	Se paga por lectura/escritura realizada y almacenamiento; con poco uso puede salir muy barato	En ambos el costo es bajo para este tamaño; RDS es predecible mes a mes
Administración	AWS se encarga del hardware, el sistema operativo y los backups automáticos; yo manejo configuración, usuarios y seguridad	AWS administra casi todo; no hay instancias que elegir en este caso	DynamoDB es más simple, pero RDS también es un servicio administrado
Ubicación y seguridad	Vive dentro de la VPC, en subredes privadas, protegida con Security Groups	Servicio regional; se accede con permisos IAM	RDS: se integra con el diseño de red que ya tengo
Mejor uso típico	Sistemas con datos relacionados y reportes (gestión escolar, ventas, finanzas)	Tráfico enorme o impredecible con consultas simples (carritos, sesiones, juegos)	RDS: Mi plataforma es un sistema de gestión estudiantil/académico.

En conclusión: RDS es mejor en estructura, relaciones/JOINs, consultas, consistencia y ubicación; hay empate en escalabilidad, rendimiento y costo; y DynamoDB gana solamente en simpleza de administración, así que en este caso y

para este proyecto en específico lo mejor sería volcarnos de por RDS.

Hablando de una justificación técnica en base a lo que se vio en la tabla, podríamos decir que elegi Amazon RDS (con MySQL) para guardar los datos de la plataforma del colegio. Mis datos son relacionales y de estructura fija: un estudiante pertenece a un curso, cada curso tiene materias y docentes, y cada entrega pertenece a un estudiante y a una materia y recibe una nota. Para armar pantallas como "las entregas de un curso con el nombre del estudiante y su nota" necesito unir varias tablas.

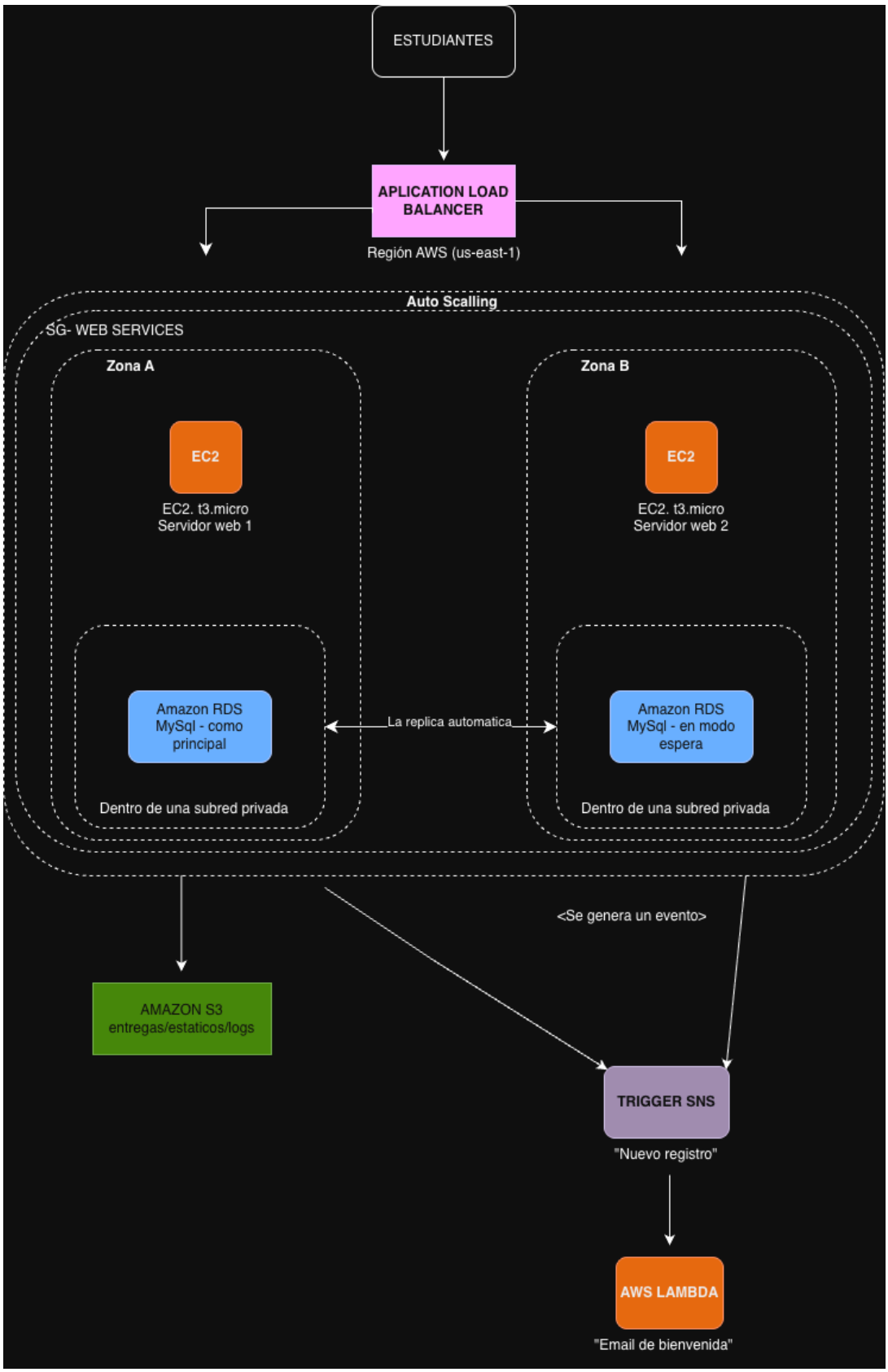
El primer beneficio es justamente ese: RDS permite JOINs y claves foráneas (FK), que evitan datos repetidos o huérfanos, por ejemplo una nota sin entrega (O null). DynamoDB no soporta JOINs, así que tendría que duplicar información o hacer muchas consultas desde la aplicación.

El segundo beneficio es la consistencia fuerte (ACID), la definición mas fundamental en las bases de datos relacionales. Las notas son información sensible y no puede pasar que se guarden a medias o que dos docentes las modifiquen a la vez con resultados distintos. Las transacciones de RDS garantizan que los datos siempre sean exactos.

El tercer beneficio es el costo acorde al tamaño. La plataforma es de un colegio, con tráfico moderado y predecible, no millones de operaciones por segundo. Una instancia chica db.t3.micro alcanza y mantiene un gasto predecible. Además los archivos pesados siguen yendo a S3 y en RDS solo guardo su clave, por lo que la base se mantiene liviana. Descarté DynamoDB porque su fortaleza (escalabilidad masiva y tráfico impredecible) no es un requisito hoy. Lo consideraría más adelante si apareciera, por ejemplo, un registro de eventos de millones de movimientos. Tampoco elegí Aurora, para no pagar de más en un proyecto inicial.

4- Diagrama de arquitectura actualizado

Actualicé el diagrama del Módulo 3 agregando la capa de datos (RDS MySQL) en subredes privadas, una en cada zona de disponibilidad. Las instancias EC2 son las únicas que se conectan a la base. También incluí Amazon S3, para optimizar el diagrama de nuestra arquitectura.


ESTUDIANTES
APLICATION LOAD BALANCER
Región AWS (us-east-1)
Auto Scalling
SG- WEB SERVICES
Zona A
Zona B
EC2
EC2. t3.micro
Servidor web 1
EC2
EC2. t3.micro
Servidor web 2
Amazon RDS MySql - como principal
La replica automatica
Amazon RDS MySql - en modo espera
Dentro de una subred privada
Dentro de una subred privada
<Se genera un evento>
AMAZON S3
entregas/estaticos/logs
TRIGGER SNS
"Nuevo registro"
AWS LAMBDA
"Email de bienvenida"

5- Medidas de seguridad para el acceso Cómputo - Datos:

Siguiendo el modelo de responsabilidad compartida: AWS se encarga de la infraestructura física y del sistema operativo de RDS, pero yo soy responsable de los datos, los accesos y la configuración de red.

Si tuvieramos que ver la politica IAM de rol en instancia ES2



La Lambda del email de bienvenida solo recibe el aviso de SNS, por lo que no necesita acceso a la base de datos.

Conclusión:

Para la plataforma del colegio técnico, RDS MySQL es la opción que mejor se ajusta porque los datos están muy relacionados, la estructura es fija, las notas exigen consistencia y el volumen es moderado. La base queda en subredes privadas y solo las EC2 pueden conectarse a ella mediante Security Groups, mientras que los archivos pesados siguen en S3.